Technical Task

Position: Senior Back-End Engineer (Java)

Expected Completion Time

This task is designed with the following expectations:

- **Senior developer:** 3–5 hours
- **Mid-level developer:** up to 12 hours
- **Junior developer:** up to 24 hours
- **Non-relevant candidate:** unlikely to complete successfully

Please keep your solution **pragmatic and focused**.

Overview

Your task is to implement a small microservice in **Java 17 or Java 21** using **Spring Boot**.

The service must expose a **REST API** and support **CRUD operations** for a **Payment domain**.

The goal is to demonstrate:

- clean backend design
- good coding practices
- production-oriented implementation

Domain: Payment

The service should manage **payments**.

A payment should include at least:

- unique identifier
- amount
- currency
- debtor account (or identifier)
- creditor account (or identifier)
- status (e.g. CREATED, COMPLETED, FAILED)

- creation timestamp

You may extend the model if needed.

Technical Requirements

Mandatory

- **Java 17 or Java 21**
- **Spring Boot**
- **Maven**
- **REST API only**

Persistence

Use **one of the following approaches**:

- **H2 embedded database**, with no additional setup required, or
- **database running in a container**

If a containerised database is used, the solution must provide **one command** that:

- builds the microservice Docker image, and
- starts all required containers so the application is ready to use

---

Functional Requirements

Your service must support:

- **Create payment**
- **Get payment by ID**
- **Update payment**
- **Delete payment**
- **List all payments**

API Specification

Implement the following REST endpoints for the **Payment** entity.

Payment fields

- `id`
- `amount`
- `currency`
- `debtorAccount`
- `creditorAccount`
- `status`

Endpoints

POST /payments
Creates a new payment.

Request example:

```
{
 "amount": 100.0,
 "currency": "EUR",
 "debtorAccount": "DE123456789",
 "creditorAccount": "DE987654321"
}
```

Response example:

```
{
 "id": "1",
 "amount": 100.0,
 "currency": "EUR",
 "debtorAccount": "DE123456789",
 "creditorAccount": "DE987654321",
 "status": "CREATED"
}
```

GET /payments/{id}
Returns a payment by ID.

PUT /payments/{id}
Updates an existing payment.

Request example:

```
{
 "amount": 150.0,
 "currency": "EUR",
```

```
 "debtorAccount": "DE123456789",
 "creditorAccount": "DE987654321"
}
```

DELETE /payments/{id}
Deletes a payment.

GET /payments
Returns all payments.

Rules

- `status` is managed by the backend
- new payment status should be `CREATED`
- `amount` must be greater than `0`
- return standard HTTP status codes

Business Logic Expectations

- Validate input data
- Implement basic status handling
- Prevent invalid updates (e.g. modifying a completed payment)

Keep the logic **simple but realistic**.

Build and Runtime

The project must include:

- **Maven build configuration**
- A **Dockerfile**

The service should be runnable locally using **Docker or Podman**.

If a containerised database is used, startup should be possible with **a single command**.

Deliverables

Please provide:

- **source code**
- **README** with:
    - how to build the project
    - how to run the service
    - how to test the API
- **Dockerfile**

Optional:

- sample requests
- brief design notes
- one-command startup (e.g. docker-compose)

Non-Functional Expectations

We are looking for:

- clean and maintainable code
- clear structure
- good separation of concerns
- meaningful error handling
- basic logging
- pragmatic decisions

Constraints

- No frontend required
- No authentication required
- No cloud deployment required
- No pagination required
- Keep the solution **simple and focused**

Submission

Please submit your solution within 24 hours of receiving the task.